

ProShop

React 19 · TypeScript · Redux Toolkit · Tailwind v4 · Vite

Complete codebase documentation — architecture, state management, feature walkthroughs, component catalogue and review findings.

Repository **proshop** · branch **main**

Generated 5 September 2026

1. Contents

#	Section
1	Contents
2	Project Overview
3	Technology Stack
4	Directory Structure
5	Application Bootstrap (main.tsx)
6	Routing (App.tsx)
7	Redux — Concepts and Wiring
8	Redux — Slice by Slice
9	The Type System (types/index.ts)
10	The Data Layer
11	Feature Walkthroughs
12	Component Catalogue
13	Styling System (Tailwind v4)
14	Responsive Strategy
15	Review Findings and Known Issues
16	Running, Building, Extending

2. Project Overview

ProShop is a fully client-side e-commerce storefront. It renders a complete shopping experience — browse, search, product detail, wishlist, cart, multi-step checkout, order confirmation, authentication, user profile, and an admin product-management back office — with **no backend server**.

Every piece of “server” data is a static JSON file imported at build time from `src/data/`. Every piece of mutable state lives in a Redux store held in browser memory. The only thing that survives a page refresh is the logged-in user, which is mirrored into `localStorage`.

What that means architecturally

- **No async anywhere.** There is no `fetch`, no `createAsyncThunk`, no loading spinners, no retry logic. Data is already in the bundle when the app starts.
- **Reducers are pure and synchronous.** Even “login” is a synchronous array lookup against `users.json`.
- **State is ephemeral by default.** Cart, wishlist, checkout form data and orders are wiped by a browser refresh. Only auth persists.
- **The design is pixel-derived.** The UI was translated from a fixed 1920 px Adobe XD canvas, so nearly every size is a literal pixel value (`text-[24px]`, `h-[62px]`, `max-w-[1640px]`) rather than a Tailwind scale token. Section 14 explains how that is made to work on smaller screens.

Mental model. Think of it as a high-fidelity prototype with real state machinery. The UI, the Redux wiring and the user flows are production-shaped; the persistence layer is a stub. Swapping `src/data/*.json` for API calls is the single change that would turn this into a real application.

3. Technology Stack

Layer	Choice	Notes
Framework	React 19.2	Function components and hooks only. No class components.
Language	TypeScript ~6.0	Strict project references: <code>tsconfig.app.json</code> + <code>tsconfig.node.json</code> .
Build tool	Vite 8	Dev server with HMR. <code>build</code> runs <code>tsc -b</code> first, so type errors fail the build.
State	Redux Toolkit 2.12 react-redux 9.3	Six slices, one store, typed hooks.
Routing	react-router-dom 7.18	<code>BrowserRouter</code> , declarative <code><Routes></code> , URL params and query params.
Styling	Tailwind CSS 4.3	CSS-first config via <code>@theme</code> . No <code>tailwind.config.js</code> file exists.
Linting	ESLint 10 + typescript-eslint	Flat config in <code>eslint.config.js</code> , with react-hooks and react-refresh plugins.
Fonts	Muli (Google Fonts)	Weights 300/400/700/900, loaded via <code><link></code> in <code>index.html</code> .
Tests	NONE	No test runner, no test files. Verification is manual.

npm scripts

```
npm run dev      # vite          - dev server, HMR
npm run build    # tsc -b && vite build
npm run lint     # eslint .
npm run preview  # vite preview  - serve the production build locally
```

4. Directory Structure

```

proshop/
|- index.html           HTML shell: #root div, Muli font links, favicon
|- vite.config.ts       plugins: react(), tailwindcss()
|- eslint.config.js     flat ESLint config
|- tsconfig*.json       project references (app + node)
|- public/assets/       product / category / hero images, illustrations
`- src/
  |- main.tsx           entry: provider tree + design-scale hack
  |- App.tsx            route table (15 routes)
  |- index.css          Tailwind import + @theme design tokens
  |
  |- components/        18 files - reusable presentational pieces
  |   Header, HeroCarousel, CategoryCard, ProductCard, ProductGrid,
  |   ProductStrip, ReviewCard, QuantityStepper, Field, Breadcrumb,
  |   StepIndicator, OrderSummaryCard, DataTable, NotFound, Icon,
  |   SearchBar (EMPTY), Button (EMPTY)
  |
  |- pages/             13 files - one per route screen
  |   HomePage, ProductPage, SearchPage, CartPage, WishlistPage,
  |   CheckoutPage, CheckoutReviewPage, OrderSuccessPage,
  |   LoginPage, SignupPage, ProfilePage,
  |   AdminProductsPage, AdminProductFormPage
  |
  |- store/             Redux: 6 slices + store + typed hooks
  |   store.ts, hooks.ts, cartSlice, wishlistSlice, checkoutSlice,
  |   orderSlice, authSlice, productsSlice
  |
  |- types/index.ts     ALL TypeScript types - single source of truth
  |
  `- data/              the "database"
      products.json, categories.json, heroSlides.json,
      reviews.json, users.json, cart.json, order.json, user.json

```

The organising rule: `pages/` know about Redux and routing. `components/` mostly do not — they take props and render. The two exceptions are `Header` and `ProductCard`, which read and write the store directly because they appear on many pages and would otherwise require prop-drilling through every one of them.

5. Application Bootstrap — src/main.tsx

Three things happen before React renders anything.

5.1 The provider tree

```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <BrowserRouter>  
      <Provider store={store}>  
        <App />  
      </Provider>  
    </BrowserRouter>  
  </StrictMode>,  
)
```

Order matters and is worth understanding:

- `StrictMode` — development only. Double-invokes renders and effects to surface impure code. This is why effects can appear to fire twice in dev; it is not a bug.
- `BrowserRouter` — provides routing context. It sits *outside* `Provider`, so router hooks (`useNavigate`, `useParams`) work anywhere inside, including in components that also use Redux.
- `Provider store={store}` — injects the Redux store into React context. Without it every `useAppSelector` call throws.

5.2 The design-scale hack

```
function applyDesignScale() {  
  const width = window.outerWidth || window.innerWidth  
  document.documentElement.style.zoom =  
    width < 768 ? '1' : String(Math.min(1, width / 1920))  
}  
applyDesignScale()  
window.addEventListener('resize', applyDesignScale)
```

This is the most unusual code in the project, and the reasoning is layered:

- 1 The design was drawn on a fixed **1920 px** canvas, so all spacing is absolute pixels. On a 1440 px laptop an unmodified layout would overflow.
- 2 Setting CSS `zoom` to `width / 1920` proportionally shrinks the whole document — one line instead of rewriting every size.
- 3 `outerWidth`, **not** `innerWidth` **or** `vw`. Browser zoom (Cmd +/-) changes `innerWidth` but not `outerWidth`. Using `outerWidth` means the app's own scaling does not fight the user's zoom.
- 4 **Disabled below 768 px.** Critical detail: CSS `zoom` changes the *effective viewport width* that Tailwind's `sm:` / `md:` media queries evaluate against. If zoom stayed active on a phone, the phone would report a wide effective viewport, match the desktop breakpoints, and render a shrunken desktop layout — unreadable. Forcing `zoom: 1` under 768 px lets phones render the genuine mobile classes at true 1:1.

Trade-off to know: `zoom` is not a standard-track property with uniform behaviour across every engine, and it is applied imperatively from JavaScript rather than declaratively in CSS. It works here because the design constraint (a fixed-pixel XD export) made the alternative — converting several hundred hard-coded pixel values to relative units — far more expensive.

6. Routing — src/App.tsx

App renders `<Header />` once, outside `<Routes>`, so the header persists across every navigation and never remounts. Then a flat route table:

Path	Component	Purpose
/	HomePage	Hero carousel, categories, featured and top-rated products
/product/:slug	ProductPage	Gallery, options, specs, reviews, add to cart
/search	SearchPage	Filter by <code>?q=</code> and/or <code>?category=</code>
/cart	CartPage	Line items, quantity, totals
/wishlist	WishlistPage	Saved items, move to cart
/checkout	CheckoutPage	Step 1 — shipping and payment forms
/checkout/review	CheckoutReviewPage	Step 2 — confirm and place order
/order/:id	OrderSuccessPage	Confirmation, guarded by order id match
/login	LoginPage	Credential form
/register	SignupPage	Registration form
/profile	ProfilePage	User details, logout
/admin/products	AdminProductsPage	Product table with edit and delete
/admin/products/new	AdminProductFormPage	Create mode
/admin/products/:id/edit	AdminProductFormPage	Edit mode (same component)
*	NotFound	Catch-all 404

Two routing techniques used

- **URL params** — `/product/:slug` read via `useParams()`. The lookup key is the human-readable `slug`, not the numeric id, which keeps URLs clean.
- **Query params** — `/search?q=...&category=...` read via `useSearchParams()`. Because search state lives in the URL rather than in Redux, search results are shareable, bookmarkable, and survive a refresh for free.

No route guards. `/admin/products` and `/admin/products/new` are reachable by typing the URL while logged out. `User.isAdmin` exists in the type and in `users.json` but is never checked by any route. `ProfilePage` is the only screen with a redirect, and it happens in an effect after render, not before.

7. Redux — Concepts and Wiring

7.1 Why Redux is here at all

Some state belongs to one screen; some belongs to the whole application. Cart contents are the clear case: the **Header** shows a badge count, **ProductCard** adds to it, **CartPage** edits it, **CheckoutPage** sums it, and **CheckoutReviewPage** clears it. Five components across four routes touching one piece of data. Passing that through props would mean threading it through every intermediate component that does not care about it. A store solves this: any component reads what it needs directly, and writes by dispatching.

State that stays local *correctly stays in* `useState` — the carousel's active slide, the product page's selected image and quantity, every form's field values. That split is a deliberate and correct judgement, not an oversight.

7.2 The three moving parts

Part	What it is
Store	One object holding all shared state. Read-only from the outside.
Action	A plain object describing an intent: <code>{ type: 'cart/addToCart', payload: {...} }</code> . Created for you by RTK.
Reducer	A pure function <code>(state, action) => newState</code> . The only thing allowed to change state.

7.3 Redux Toolkit's `createSlice`

Classic Redux required action-type string constants, action-creator functions, and a `switch`-based reducer — three files of boilerplate per feature. `createSlice` collapses all of it:

```
const cartSlice = createSlice({
  name: 'cart', // becomes the action-type prefix
  initialState,
  reducers: {
    removeFromCart(state, action: PayloadAction<string>) {
      state.lines = state.lines.filter(l => l.id !== action.payload)
    },
  },
})

export const { removeFromCart } = cartSlice.actions // auto-generated creators
export default cartSlice.reducer
```

From that single object RTK derives the action type string `'cart/removeFromCart'`, the action creator `removeFromCart(id)`, and the reducer function.

7.4 Immer — why mutation looks legal here

This line appears in `cartSlice` and looks like it breaks Redux's first rule:

```
existing.qty += qty // direct mutation?!
```

It does not. RTK wraps every reducer in **Immer**. The `state` you receive is a *draft proxy*, not the real state. Immer records the mutations you perform on the draft and produces a brand-new immutable state object from them. You get readable mutable-looking code and correct immutable semantics at the same time.

Why immutability matters: react-redux decides whether to re-render by comparing the value your selector returned before and after, using `===`. If a reducer mutated the real state object in place, the reference would be identical, the comparison would say “unchanged”, and the UI would silently fail to update. Immer guarantees a fresh reference whenever something actually changed.

7.5 Store assembly — store/store.ts

```
export const store = configureStore({
  reducer: {
    cart: cartReducer,
    checkout: checkoutReducer,
    order: orderReducer,
    auth: authReducer,
    products: productsReducer,
    wishlist: wishlistReducer,
  },
})
```

The keys of that object become the shape of the state tree, so `state.cart.lines`, `state.auth.user` and so on. `configureStore` also enables Redux DevTools and installs development middleware that throws if you accidentally mutate state outside a reducer or put a non-serialisable value in the store.

7.6 Typed hooks — store/hooks.ts

```
export const useAppDispatch: () => AppDispatch = useDispatch
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector
```

Twelve tokens that pay for themselves constantly. `RootState` is derived from the store itself (`ReturnType<typeof store.getState>`), so it can never drift out of sync with the actual reducers. The result: full autocomplete inside every selector, and `state.cart.linez` becomes a compile error rather than a runtime `undefined`.

Convention to follow: always import `useAppSelector` / `useAppDispatch` from `store/hooks`. Never import raw `useSelector` / `useDispatch` from `react-redux` — you would lose every type guarantee above.

7.7 Selector granularity

Note how `Header` subscribes:

```
const itemCount = useAppSelector(
  (state) => state.cart.lines.reduce((sum, l) => sum + l.qty, 0)
)
```

It selects a **number**, not the array. The component re-renders only when the computed total actually changes value. Selecting `state.cart.lines` instead would re-render the header on any cart change at all. Selecting an object or array literal (`{ a: state.x, b: state.y }`) would be worse still — a new reference every time, so `===` always fails and the component re-renders on every dispatched action anywhere in the app. Every selector in this codebase returns either a primitive or a stable existing reference.

8. Redux — Slice by Slice

8.1 cartSlice — state.cart

Shape: { lines: CartLine[] }

Action	Payload	Behaviour
addToCart	{ item, qty? }	If a line with that id exists, increment its qty; otherwise push a new line. qty defaults to 1.
removeFromCart	string(id)	Filter the line out.
updateQty	{ id, qty }	Set an exact quantity (used by QuantityStepper).

The merge behaviour in `addToCart` is the important detail — adding the same product twice produces one line of quantity 2, not two lines. A `CartLine` stores a *snapshot* of the product (id, slug, name, price, image), not a reference to it. That is intentional: a cart should record the price at the moment of adding, so a later price change does not silently rewrite what the customer thought they were buying.

8.2 wishlistSlice — state.wishlist

Shape: { items: WishlistItem[] }

```
toggleWishlist(state, action: PayloadAction<WishlistItem>) {
  const exists = state.items.some(i => i.id === action.payload.id)
  state.items = exists
    ? state.items.filter(i => i.id !== action.payload.id)
    : [...state.items, action.payload]
}
```

One action for both directions. The heart button in `ProductCard` and `ProductPage` does not need to know the current state before dispatching — it just toggles, and the reducer decides. That keeps the components dumb.

`removeFromWishlist` exists separately for the explicit X button on `WishlistPage`, where “toggle” would be the wrong semantic.

8.3 checkoutSlice — state.checkout

Shape: { shipping: ShippingAddress, payment: PaymentDetails }

Two setters, `setShipping` and `setPayment`. This slice exists for one reason: to carry form data *across a route change* from `/checkout` to `/checkout/review`. Local `useState` would be destroyed when `CheckoutPage` unmounts.

The round-trip is designed deliberately. `CheckoutPage` seeds its local state from the store (`useState(savedShipping.country)`) and writes back on submit. So pressing “Change” on the review page and returning to the form shows the values already filled in, rather than an empty form.

8.4 orderSlice — state.order

Shape: { lastOrder: Order | null }. One action, `placeOrder`.

Only the most recent order is retained — hence the name. `OrderSuccessPage` uses it as a guard:

```
if (!order || order.id !== id) return <NotFound message="Order not found." />
```

The URL id must match the stored order, so a fabricated `/order/ABC123` shows the 404 state rather than a blank or broken confirmation screen.

8.5 authSlice — state.auth

Shape: { user: User | null, error: string | null }

The only slice with persistence:

```
const STORAGE_KEY = 'proshop_user'

function loadUser(): User | null {
  const raw = localStorage.getItem(STORAGE_KEY)
  return raw ? JSON.parse(raw) : null
}

const initialState: AuthState = { user: loadUser(), error: null }
```

`loadUser()` runs once at module evaluation, so the session is restored before React renders and there is no logged-out flash on refresh.

The security-relevant detail in login

```
const match = usersData.find(
  (u) => u.email === action.payload.email && u.password === action.payload.password
)
if (!match) { state.error = 'Invalid email or password'; return }

const user: User = { id: match.id, name: match.name,
  email: match.email, isAdmin: match.isAdmin }
```

The matched record is **not** spread into state. Four fields are copied explicitly, which excludes `password`. If the code had used `{ ...match }`, the password would land in the Redux store, in Redux DevTools, and — via the same object — in `localStorage`, readable by any script on the page. Explicit field selection makes that leak impossible by construction.

This is prototype auth, not real auth. Passwords are plaintext in `users.json` and shipped inside the JavaScript bundle. `register` creates a session but never persists the new user, so a registered account cannot log in again after logout. There is no token, no expiry, no server verification. Adequate for a demo; must be replaced entirely before any real deployment.

8.6 productsSlice — state.products

Shape: { items: Product[] }, seeded from `products.json`.

Action	Behaviour
<code>addProduct</code>	<code>unshift</code> — new products appear at the top of the admin table.
<code>updateProduct</code>	<code>findIndex</code> by id, replace in place.
<code>deleteProduct</code>	Filter out by id.

This is the store's only genuine CRUD surface, and it powers the admin back office. See Section 15 for the significant caveat about who actually reads from it.

9. The Type System — src/types/index.ts

Every type in the project lives in this one file, organised into four labelled regions:

- 1 **Domain** — `Product`, `CartLine`, `WishlistItem`, `ShippingAddress`, `PaymentDetails`, `Order`, `User`
- 2 **Store** — one `*State` type per slice, plus `RootState` and `AppDispatch`
- 3 **Component props** — one `*Props` type per component
- 4 **Generics** — `Column<T>` and `DataTableProps<T>`

Why centralise

Types describing the same concept were previously declared in several components and slices, which allowed them to drift. One file means one definition of `Product`; changing it surfaces every affected call site as a compile error immediately.

The generic table type

```
export type Column<T> = {
  key: string
  label: string
  render?: (row: T) => ReactNode
}
```

Used as `<DataTable<Product> ... />`, so inside every `render` callback the `row` argument is fully typed as a `Product` with working autocomplete. The component stays reusable for any row type without resorting to `any`.

Known gap. `Product` declares eleven fields but omits `gallery`, `options` and `specs` — all three of which exist in `products.json` and are read by `ProductPage`. It compiles only because `ProductPage` imports the JSON directly (TypeScript infers the richer literal shape) while `productsSlice` casts it with as `Product[]`, discarding those fields. Section 15 covers the consequences.

10. The Data Layer

File	Contents
<code>products.json</code>	Catalogue. Each product: <code>id</code> , <code>slug</code> , <code>name</code> , <code>brand</code> , <code>category</code> , <code>price</code> , <code>rating</code> , <code>reviewCount</code> , <code>countInStock</code> , <code>image</code> , <code>description</code> , plus optional <code>gallery[]</code> , <code>options[]</code> , <code>specs[]</code> .
<code>categories.json</code>	Four cards: <code>label</code> , <code>image</code> , and an <code>href</code> pre-built as <code>/search?category=...</code>
<code>heroSlides.json</code>	Carousel slides: <code>eyebrow</code> , <code>headline</code> , <code>body</code> , <code>cta</code> , <code>href</code> , <code>image</code> .
<code>reviews.json</code>	Reviews keyed by <code>productId</code> , filtered client-side on the product page.
<code>users.json</code>	Two demo accounts. <code>a@b / 123</code> is admin; <code>m@m / 123</code> is a regular user.
<code>cart.json</code> , <code>order.json</code> , <code>user.json</code>	UNUSED. Legacy fixtures superseded by the Redux slices.

Because these are ES module imports rather than network requests, Vite inlines them into the bundle at build time. That is why nothing in the app is asynchronous.

11. Feature Walkthroughs

11.1 Browsing and search

HomePage composes four sections: `HeroCarousel`, a category grid, “Featured Products” (`products.slice(0, 3)`) and “Top Rate Products” (`products.slice(3, 6)`). The featured/top-rated split is positional, not computed from ratings.

Search reads both query parameters and filters in one pass:

```
const results = products.filter((p) => {
  const matchesQuery = !q ||
    p.name.toLowerCase().includes(q) || p.brand.toLowerCase().includes(q)
  const matchesCategory = !category || p.category === category
  return matchesQuery && matchesCategory
})
```

Each condition short-circuits to `true` when its parameter is absent, so the same expression handles text-only search, category-only browsing (the category cards link straight here), both together, and neither. The header's search form navigates to `/search?q=...` with `encodeURIComponent`, and seeds its input from the current `?q=` so the term stays visible on the results page.

11.2 Product detail

ProductPage looks the product up by slug and renders `NotFound` if it is missing. It manages three pieces of local state:

- `activeImage` — index into `gallery`, driven by the thumbnail strip (falls back to `[product.image]` when no gallery exists)
- `qty` — passed to `QuantityStepper`, capped at `countInStock`
- `selected` — a `Record<string, string>` of chosen option values, keyed by option label

Options render as colour swatches when the label is `"Color"` and as pill buttons otherwise.

Selected options are dropped on add-to-cart. The dispatch sends only id, slug, name, price, image and quantity. Choosing “512GB” changes the UI but not what is bought, and the price never varies by option. `CartLine` would need an `options` field to fix this properly.

11.3 Cart

Two distinct renders: an empty state (message, “Keep Shopping” button, “Recently viewed” strip) and the populated grid. Each line offers a `QuantityStepper` (dispatching `updateQty`), a live line total, and a remove button. The right-hand column shows the summary and the checkout link.

11.4 Wishlist

Mirrors the cart's structure. The heart button on `ProductCard` and `ProductPage` dispatches `toggleWishlist` and restyles itself from the store (`isWishlisted ? 'bg-brand border-brand' : 'border-surface'`). The header badge shows `wishlist.items.length`. On the wishlist page each row offers “Add to cart” — which dispatches `addToCart({ item })` directly, since `WishlistItem` is structurally exactly `Omit<CartLine, 'qty'>`. No mapping needed; the types line up by design.

11.5 Checkout — the full sequence

- 1 **/checkout** — nine `Field` components, seeded from `state.checkout`. The order summary computes subtotal, 1% tax and zero shipping live. “Review order” dispatches `setShipping` and `setPayment`, then navigates.

- 2 **/checkout/review** — reads shipping, payment and cart lines from the store. The card number is masked to ****1234. Each block has a “Change” button routing back to step 1, where the saved values reappear.
- 3 **Place Order** — generates a random 16-character alphanumeric id, dispatches `placeOrder` with a full `Order` snapshot, empties the cart by dispatching `removeFromCart` per line, then navigates to `/order/:id`.
- 4 **/order/:id** — validates the id against `state.order.lastOrder` and renders the confirmation, or `NotFound`.

`StepIndicator` receives `current={1}` and `current={2}` on the two pages to highlight progress.

11.6 Authentication

Both auth pages follow the same pattern: local `useState` per field, dispatch on submit, and an effect that redirects once `state.auth.user` becomes non-null:

```
useEffect(() => { if (user) navigate('/profile') }, [user, navigate])
```

The reducer never navigates. It sets state; the effect observes the change and reacts. That keeps the reducer pure and testable.

`SignupPage` splits error sources: `formError` for client-side validation (password mismatch) and `authError` from the store (email already registered), merged for display with `formError || authError`. `ProfilePage` guards in reverse — redirect when `user` is `null` — and returns `null` during the render before the effect fires, which prevents a crash on `user.name`.

11.7 Admin back office

`AdminProductsPage` feeds `state.products.items` into the generic `DataTable`, with a custom `render` for the price (formatted) and for the action column (edit link plus delete button behind a `window.confirm`).

`AdminProductFormPage` serves create and edit from one component, distinguished by whether `useParams()` yields an id. Details worth noting:

- A generic setter keeps the field wiring type-safe: `function set<K extends keyof Product>(key: K, value: Product[K])`.
- Slug auto-generation: `form.name.toLowerCase().trim().replace(/\s+/g, '-')` when the field is left blank.
- Image upload uses `URL.createObjectURL(file)` — an in-memory blob URL. It previews correctly but is invalidated on page reload, since there is nowhere to upload to.

12. Component Catalogue

Component	Store	Responsibility
Header	reads	Sticky nav. Cart and wishlist badges, auth-aware user block, search form.
HeroCarousel	—	Slide state with wrap-around arrows and dot navigation. <code>min-h</code> per text block prevents layout jump between slides.
CategoryCard	—	Image plus label linking into a filtered search.
ProductCard	reads + writes	Grid tile. Discount badge, star row, add-to-cart and wishlist toggle.
ProductGrid	—	Responsive 1 / 2 / 3-column grid of <code>ProductCard</code> .
ProductStrip	—	Horizontal three-up band used for “Recently viewed”.
ReviewCard	—	Author, date, stars, body.
QuantityStepper	—	Controlled <code>-/+</code> stepper, clamped with <code>Math.max(min, ...)</code> and <code>Math.min(max, ...)</code> .
Field	—	Labelled input. Simplified <code>onChange: (value: string) => void</code> instead of an event.
OrderSummaryCard	—	Subtotal / tax / shipping / total plus a configurable CTA. Used by both checkout steps.
StepIndicator	—	Highlights the current step from a steps array.
DataTable<T>	—	Generic table: column definitions with optional custom renderers, zebra rows, horizontal scroll wrapper.
NotFound	—	Reusable empty/404 state with configurable message and back link.
Breadcrumb	—	“Back / {label}” text.
Icon	—	Four inline SVGs (User, Wishlist, Search, Cart) using <code>fill="currentColor"</code> so they inherit text colour.
SearchBar, Button	—	EMPTY FILES. Zero bytes. Nothing imports them. Safe to delete.

13. Styling System — Tailwind v4

Tailwind v4 configures through CSS, not JavaScript. The entire design system is `src/index.css`:

```
@import "tailwindcss";

@theme {
  --color-brand:      #FCDD06; /* signature yellow */
  --color-ink:        #242424; /* near-black text and header */
  --color-surface:     #F2F2F2; /* card grey */
  --color-surface-alt: #F7F8FC; /* image-well grey */
  --color-danger:     #FC4059; /* errors, discount badge, delete */
  --color-muted:      #707070; /* secondary text */
  --color-grey-300:   #C7C7C7; /* borders */
  --font-sans: "Muli", system-ui, sans-serif;
  --radius-card: 16px; --radius-btn: 10px;
  --radius-pill: 20px; --radius-input: 6px;
}
```

Each custom property generates utilities automatically: `--color-brand` yields `bg-brand`, `text-brand`, `border-brand`; `--radius-card` yields `rounded-card`. There is no `tailwind.config.js` and none is needed.

Conventions in use

- **Semantic names over literal ones.** `bg-surface`, not `bg-[#F2F2F2]`. Rebranding means editing seven lines.
- **Arbitrary values for XD-derived sizes.** `text-[24px]`, `h-[62px]`, `max-w-[1640px]` — traceable straight back to the design file.
- **One page shell**, repeated on every page: `max-w-[1640px] mx-auto px-6 py-10`.

14. Responsive Strategy

Two mechanisms cooperate, split at the `md` breakpoint (768 px):

Viewport	Mechanism
≥ 768 px	CSS zoom scales the 1920 px design down proportionally. The layout is structurally identical to the desktop design, only smaller.
< 768 px	zoom forced to 1. Tailwind's responsive utilities take over and produce a genuinely different mobile layout.

Mobile-specific patterns found throughout

- **Progressive type scale** — `text-[22px]` `sm:text-[26px]` `md:text-[32px]` applied consistently to every page heading.
- **Axis flip** — `flex flex-col sm:flex-row` on cart and wishlist rows: stacked on phones, horizontal on tablets and up.
- **Label hiding** — `hidden sm:inline` on header nav labels, leaving icons alone on narrow screens.
- **Search reflow** — `order-3 md:order-none basis-full md:basis-auto` drops the search field onto its own full-width row below the logo on phones.
- `sm:contents` in the cart row — collapses a mobile-only wrapper `div` out of the box model at `sm` and up, so its children join the parent flex row directly. One markup tree, two layouts, no duplication.
- **Grid ramps** — `grid-cols-1 sm:grid-cols-2 lg:grid-cols-3` for products; `grid-cols-2 sm:grid-cols-3 lg:grid-cols-4` for categories.

15. Review Findings and Known Issues

HIGH — Cart total subtracts tax

src/pages/CartPage.tsx:14

```
const tax    = subtotal * 0.01
const total = subtotal - tax          // CartPage          -> $99 on a $100 cart
// but
const total = subtotal + tax + shippingCost // Checkout & Review -> $101
```

The cart page shows a total one percent *below* subtotal; the checkout pages show one percent above. On a \$100 cart the customer sees \$99, then is charged \$101.

Root cause is duplication, not the sign. The same subtotal/tax computation is written three times — `CartPage`, `CheckoutPage`, `CheckoutReviewPage` — with `0.01` hard-coded in each. Three copies drifted; one of them drifted wrong.

Fix at the fork:

```
// one shared helper, all three pages call it
const TAX_RATE = 0.01

export function selectCartTotals(lines: CartLine[], shippingCost = 0) {
  const subtotal = lines.reduce((sum, l) => sum + l.price * l.qty, 0)
  const tax = subtotal * TAX_RATE
  return { subtotal, tax, shippingCost, total: subtotal + tax + shippingCost }
}
```

After this, cart and checkout cannot disagree — not because someone remembers to keep them in sync, but because there is only one place to be wrong.

HIGH — Admin edits are invisible on the storefront

The storefront pages import the JSON module directly:

```
import products from '../data/products.json' // HomePage, ProductPage,
                                              // SearchPage, CartPage,
                                              // WishlistPage, OrderSuccessPage
```

Only `AdminProductsPage` and `AdminProductFormPage` read `state.products.items`. So creating, editing or deleting a product updates the admin table and nothing else — the home page, search and product detail continue serving the original JSON.

Fix: replace every storefront JSON import with `useAppSelector((s) => s.products.items)`. The slice already holds the seeded data, so this is a substitution, not new machinery.

MEDIUM — Full page reloads break the SPA

`ProductCard`, `ProductStrip`, `CategoryCard` and the `HeroCarousel` CTA use raw `<a href>` instead of react-router's `<Link to>`. Every such click triggers a full browser navigation: the app restarts and the entire Redux store — cart, wishlist, checkout data, order — is wiped. Only auth survives, because it is in `localStorage`.

Fix: swap to `<Link to={...}>` in those four components. Inside `ProductCard`, keep the existing `e.preventDefault()` / `e.stopPropagation()` calls on the two nested buttons so they do not trigger the surrounding link.

MEDIUM — Admin routes are unprotected

`User.isAdmin` is defined, populated in `users.json`, and stored on login — but never read anywhere. Navigating directly to `/admin/products` while logged out shows the full admin table with working delete buttons.

Fix: a small wrapper component that reads `state.auth.user` and returns `<Navigate to="/login" replace />` unless `user?.isAdmin`, wrapped around the three admin routes.

LOW — Assorted

- **Incomplete Product type** — missing `gallery`, `options`, `specs`. Because `productsSlice` casts with `as Product[]`, any product created through the admin form can never carry a gallery, options or specs.
- `register` **does not persist** — a new account works for the current session, but `users.json` is immutable, so logging out loses the account permanently.
- **Cart discards selected options** — see §11.2.
- **Cart and wishlist are not persisted** — a refresh empties both. If desired, the `localStorage` pattern already proven in `authSlice` extends directly.
- **Two empty files** — `components/SearchBar.tsx` and `components/Button.tsx` are zero bytes and unreferenced. Delete them.
- **Three unused data files** — `cart.json`, `order.json`, `user.json`. Superseded by the slices.
- **Nested interactive elements** — `ProductCard` puts two `<button>` elements inside an anchor. It works via event handling, but it is invalid HTML nesting and confuses screen readers and keyboard navigation.
- **No tests** — no runner, no test files. The tax bug above is precisely the class of defect a single unit test on a shared totals helper would have caught.

16. Running, Building, Extending

Local development

```
npm install
npm run dev          # http://localhost:5173

# demo credentials (src/data/users.json)
#   admin    a@b / 123
#   user     m@m / 123
```

Verification before committing

```
npm run lint          # eslint .
npm run build          # tsc -b && vite build - type errors fail the build
```

There is no test suite, so these two commands are the whole safety net. Treat a clean `build` as necessary but not sufficient — the tax bug in §15 type-checks perfectly.

Adding a new piece of global state

- 1 Define its `State` type in `src/types/index.ts`.
- 2 Create `src/store/xSlice.ts` with `createSlice`; export the generated actions and the reducer.
- 3 Register the reducer in `src/store/store.ts`. `RootState` updates automatically — it is derived from the store.
- 4 Consume with `useAppSelector` / `useAppDispatch`, selecting the narrowest value the component actually needs (see §7.7).

Adding a page

- 1 Create `src/pages/XPage.tsx`, opening with the standard shell `<div className="max-w-[1640px] mx-auto px-6 py-10">`.
- 2 Import it in `src/App.tsx` and add a `<Route>` above the `path="*" catch-all`.
- 3 Navigate to it with `<Link to>`, never `<a href>` — see §15.

Replacing the JSON layer with a real API

The migration is contained because every data read is already funnelled through a slice or a direct import:

- 1 Point all storefront pages at `state.products.items` (this is the §15 fix, and it is a prerequisite).
- 2 Replace each slice's seeded `initialState` with a `createAsyncThunk` that fetches, and add `loading` / `error` fields to the corresponding `*State` type.
- 3 Move authentication server-side: hash passwords, issue a token, store the token rather than the user object, and verify `isAdmin` on the server for every admin operation — never trust the client-side flag.
- 4 Introduce the shared totals helper (§15) and, ultimately, compute authoritative totals on the server; client-side totals become display-only.

Closing note. The architecture here is sound: clean separation between pages and components, correct instinct about which state is global and which is local, disciplined selector granularity, a genuinely reusable generic table, centralised types, and a real security-conscious choice in the login reducer. The defects that do exist cluster in one place — duplicated logic that drifted (the tax calculation, and the JSON-versus-store split). Both are fixed by consolidating to a single source of truth, which is the same principle `types/index.ts` already applies successfully to types.